For example, the bubble sort algorithm can be used to sort a list of ten temperatures stored in the array, **`Temperature[]`**, into ascending order. It could be written in pseudocode as:

```
First ← 1
Last ← 10
REPEAT
  Swap ← FALSE
  FOR Index ← First TO Last - 1
    IF Temperature[Index] > temperature[Index + 1]
      THEN
        Temp ← Temperature[Index]
        Temperature[Index] ← Temperature[Index + 1]
        Temperature[Index + 1] ← Temp
        Swap ← TRUE
    ENDIF
  NEXT Index
  Last ← Last - 1
UNTIL (NOT Swap) OR Last = 1
```

The `IF..THEN` condition checks if temperatures are in ascending order and swaps them if they are not, using the Temp variable (short for temporary)

7.5 Validation and verification

In order for computer systems to only accept data inputs that are reasonable and accurate, every item of data needs to be examined before it is accepted by the system.

Two different methods, with very similar sounding names, are used. For data entry, validation ensures that only data that is reasonable is accepted. Verification is used to check that the data does not change as it is being entered.

7.5.1 Validation

Validation is the automated checking by a program that data is reasonable before it is accepted into a computer system. When data is validated by a computer system, if the data is rejected a message should be output explaining why the data was rejected and another opportunity given to enter the data.

There are many different types of validation checks including:

- **range checks**
- **length checks**
- **type checks**
- **presence checks**
- **format checks**
- **check digits.**

Different types of check may be used on the same piece of data; for example, an examination mark could be checked for reasonableness by using a range check, a type check and a presence check.

Range check

A **range check** checks that the value of a number is between an upper value and a lower value. For example, checking that percentage marks are between 0 and 100 inclusive:

```
OUTPUT "Please enter the student's mark "
REPEAT
INPUT StudentMark
IF StudentMark < 0 OR StudentMark > 100
  THEN
    OUTPUT "The student's mark should be in the range
            0 to 100, please re-enter the mark "
ENDIF
UNTIL StudentMark >= 0 AND StudentMark <= 100
```

Length check

A **length check** checks *either*:

- that data contains an exact number of characters, for example that a password must be exactly eight characters in length so that passwords with seven or fewer characters or nine or more characters would be rejected, for instance:

Password has a data type of string and LENGTH is the pseudocode operation that returns a whole number showing the number of characters in the string

```
OUTPUT "Please enter your password of eight
characters "
REPEAT
    INPUT Password
    IF LENGTH(Password) <> 8
      THEN
        OUTPUT "Your password must be exactly eight
                characters, please re-enter "
    ENDIF
UNTIL LENGTH(Password) = 8
```

- *or* that the data entered is a reasonable number of characters, for example, a family name could be between two and thirty characters inclusive so that names with one character or thirty-one or more characters would be rejected.

FamilyName has a data type of string and LENGTH is the pseudocode operation that returns a whole number showing the number of characters in the string

```
OUTPUT "Please enter your family name "
REPEAT
  INPUT FamilyName
  IF LENGTH(FamilyName) > 30 OR LENGTH(FamilyName) < 2
    THEN
      OUTPUT "Too short or too long,
      please re-enter "
  ENDIF
UNTIL LENGTH(FamilyName) <= 30 AND LENGTH(FamilyName) >= 2
```

Link

To understand some of the concepts and commands in this code, such as **DIV**, see Chapter 8.

Find out more

Programming is covered in Chapter 8. When you have started your programming, find out how you could test for a whole number and write and test a program for this validation rule.

Type check

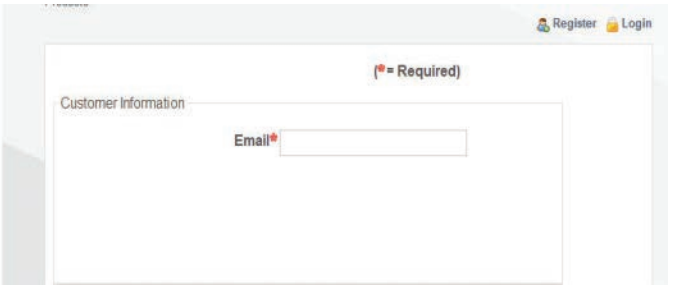
A type check checks that the data entered is of a given data type, for example, that the number of brothers or sisters would be an integer (whole number).

```
OUTPUT "How many brothers do you have? "
REPEAT
INPUT NumberOfBrothers
IF NumberOfBrothers <> DIV(NumberOfBrothers, 1)
  THEN
    OUTPUT "This must be a whole number, please re-enter"
ENDIF
UNTIL NumberOfBrothers = DIV(NumberOfBrothers, 1)
```

Presence check

A **presence check** checks to ensure that some data has been entered and the value has not been left blank, for example, an email address for an online transaction must be completed.

```
OUTPUT "Please enter your email address "
REPEAT
INPUT EmailAddress
IF EmailAddress = ""
  THEN
   OUTPUT "*=Required "
ENDIF
UNTIL EmailAddress <> ""
```



▲ **Figure 7.12** Presence check error message

Format check and check digit

A **format check** checks that the characters entered conform to a pre-defined pattern, for example, in Chapter 9 the cub number must be in the form CUB9999. The pseudocode for this example will be given in the string handling section of Chapter 9.

A **check digit** is the final digit included in a code; it is calculated from all the other digits in the code. Check digits are used for barcodes, product codes, International Standard Book Numbers (ISBN) and Vehicle Identification Numbers (VIN).

Link

ISBN and modulo-11 check digit calculations are covered in Chapter 2.

Check digits are used to identify errors in data entry caused by mis-typing or mis-scanning a barcode. They can usually detect the following types of error:

- an incorrect digit entered, for example, 5327 entered instead of 5307
- transposition errors where two numbers have changed order for example 5037 instead of 5307
- omitted or extra digits, for example, 537 instead of 5307 or 53107 instead of 5307
- phonetic errors, for example, 13, thirteen, instead of 30, thirty.



▲ **Figure 7.13** ISBN 13 code with check digit

Find out more

1. Find an ISBN, then show that its check digit is correct.
   Working in pairs find two ISBNs each, copy one down with a transposition error and the other one correctly.
   Swap your ISBNs and see if you can find the one with the error.
   Look at a correct ISBN, can you think of an error that this system will not identify and explain with an example why this is the case?
2. Find out how limit checks and consistency checks are used.

Activity 7.7

1 State, with reasons, which validation checks you could use for the following inputs.

You may decide that more than one validation check is required.
- Entering a telephone number
- Entering a pupil's name
- Entering a part number in the form XXX999, when X must be a letter and 9 must be a digit.

2 Write an algorithm using pseudocode to check the age and height of a child who wants to go on a fairground ride. The age must be over 7 and under 12, the height must be over 110 centimetres and under 150 centimetres.

3 Write an algorithm using pseudocode to check that the length of a password is between 8 and 12 characters inclusive.

7.5.2 Verification

Verification is checking that data has been accurately copied from one source to another – for instance, input into a computer or transferred from one part of a computer system to another.

Verification methods for input data include:

- Double entry
- Screen/visual check.

For **double entry** the data is entered twice, sometimes by different operators. The computer system compares both entries and if they are different outputs an error message requesting that the data is entered again.



▲ **Figure 7.14** Double entry

A **screen/visual check** is a manual check completed by the user who is entering the data. When the data entry is complete the data is displayed on the screen and the user is asked to confirm that it is correct before continuing. The user either checks the data on the screen against a paper document that is being used as an input form or, confirms whether it is correct from their own knowledge.

Link

Parity checks and checksums are used when data is transferred from one part of a computer system to another, or across a network, and are discussed in Chapter 2.

Activity 7.8

Explain why the following input data also needs to be verified:

- Entering a telephone number
- Entering a pupil's name
- Entering a part number in the form XXX999, when X must be a letter and 9 must be a digit.

7.6 Test data

7.6.1 How to suggest and apply suitable test data

In order to determine whether a solution is working as it should, it needs to be tested. Usually before a whole system is tested each sub-system is tested separately.

Algorithms written in pseudocode or as flowcharts can be tested by a person working through them using any data that is required and seeing what the result is. Computer programs can be tested by running them on a computer using any data that is required and seeing what result is output. However, in order to test a solution thoroughly it may need to be worked through several times with different sets of test data.

A **set of test data** is all the items of data required to work through a solution. For instance, the set of test data used for Activity 7.6 was 7 and 18.

In order to prove that program or algorithm solutions do what they are supposed to do, a set of test data should be used that the program would normally be expected to work with, together with the result(s) that are expected from that data. The type of test data used to do this is called **normal data**. Normal data should be used to work through the solution to find the actual result(s) and see if they are the same as the expected result(s).

For example, consider an algorithm that records the percentage marks, entered in whole numbers, from ten end-of-term examinations for a pupil, and then finds the average mark. A set of normal test data for this purpose could be:

Normal test data: 50, 50, 50, 50, 50, 50 50, 50, 50, 50

Expected result: 50

Activity 7.9

Provide a more realistic set of test data and its expected result.

Solutions also need to be tested to prove that they do **not** do what they are supposed not to do. In order to do this, test data should be chosen that would be rejected by the solution as not suitable, if the solution is working properly. This type of test data is called **abnormal test data**. (It is also sometimes called **erroneous test data**.)

For example, erroneous/abnormal data for our algorithm to find the average percentage marks from ten end of term examinations could be:

Erroneous/abnormal data: -12, eleven

Expected results: both of these values should be rejected

Activity 7.10

Provide some more erroneous/abnormal data and its expected results.

When testing algorithms with numerical values, sometimes only a given range of values should be allowed. For example, percentage marks should only be in the range 0 to 100. Our algorithm above should be tested with **extreme data**. Extreme data are the largest and smallest values that normal data can take. In this case:

Extreme data: 0, 100

Expected results: these values should be accepted

There is another type of test data called **boundary data**. This is used to establish where the largest and smallest values occur. At each boundary two values are required: one value is accepted and the other value is rejected. For example, for percentage marks in the range 0 to 100, the algorithm should be tested with the following **boundary data**:

Boundary data for 0 is: -1, 0

Expected results: -1 is rejected, 0 is accepted

Activity 7.11

1 Provide boundary data for the upper end of the range; assume that the percentage marks are always whole numbers.

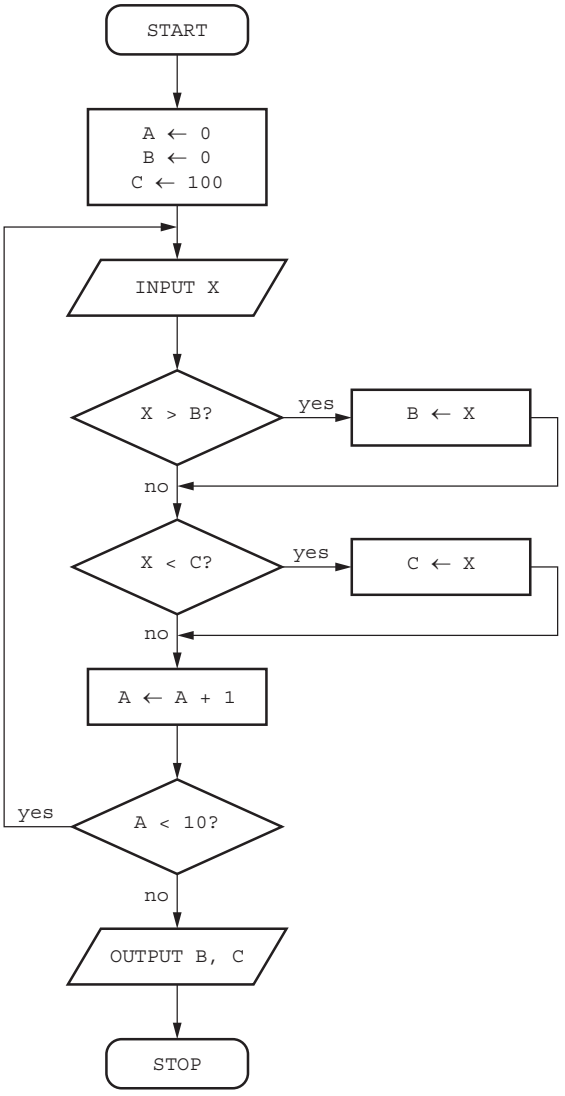
2 The end of term examinations are now marked out of twenty. Provide the following:
- Two sets of normal data and their expected results
- Some erroneous/abnormal data and their expected results
- Two sets of boundary data and their expected results.

7.7 Trace tables to document dry runs of algorithms

A thorough structured approach is required to find out the purpose of an algorithm. This involves recording and studying the results from each step in the algorithm and requires the use of test data.

Worked example

Consider the algorithm represented by the following flowchart:



▲ **Figure 7.15** Flowchart to trace

A **trace table** can be used to record the results from each step in an algorithm; it is used to record the value of an item (variable) each time that it changes. The manual exercise of working through an algorithm step by step is called a **dry run**.

A trace table is set up with a column for each variable and a column for any output. For example:

▼ **Table 7.3** Trace table

A	B	C	X	OUTPUT
0	0	100		

Test data is then used to dry run the flowchart and record the results on the trace table. During a dry run:

- every time the value of a variable is changed, the new value is entered in that column of the trace table
- every time a value is output, the value is shown in the output column.

Test data: 9, 7, 3, 12, 6, 4, 15, 2, 8, 5

▼ **Table 7.4** Completed trace table for flowchart

A	B	C	X	OUTPUT
0	0	100		
1	9	9	9	
2		7	7	
3		3	3	
4	12		12	
5			6	
6			4	
7	15		15	
8		2	2	
9			8	
10			5	
				15 2

The values 15 and 2 without the comma

Activity 7.12

Use a trace table and the test data 400, 800, 190, 170, 300, 110, 600, 150, 130, 900 to record another dry run of the highest and lowest number flowchart from Section 7.7.

It can be seen from the output that the algorithm selects the largest and the smallest numbers from a list of ten positive numbers. The same trace table could have been used if the algorithm had been shown using pseudocode:

```
A ← 0
B ← 0
C ← 100
OUTPUT "Enter your ten values"
REPEAT
  INPUT X
  IF X > B
    THEN
      B ← X
  ENDIF
  IF X < C
    THEN
      C ← X
  ENDIF
  A ← A + 1
UNTIL A = 10
OUTPUT B, C
```

Often questions will use variables with a single letter, instead of a meaningful identifier, and could ask what the purpose of the algorithm is!

This prompt needs to be shown as the first output. No quotation marks should be shown in the output column

Activity 7.13

Use the trace table below and the test data 4, 8, 19, 17, 3, 11, 6, 1, 13, 9 to record a dry run of the pseudocode.

▼ **Table 7.5** Trace table to complete for the pseudocode

A	B	C	X	OUTPUT
0	0	100		10

Activity 7.14

Draw a trace table for the bubble sort algorithm on page 276 and use the test data 35, 31, 32, 36, 39, 37, 42, 38 to record a dry run of the pseudocode.

7.8 Identifying errors in algorithms

Trace tables and test data can be used to identify and correct errors.

Your completed trace table for Activity 7.14 should look like this:

▼ **Table 7.6** Completed trace table for flowchart

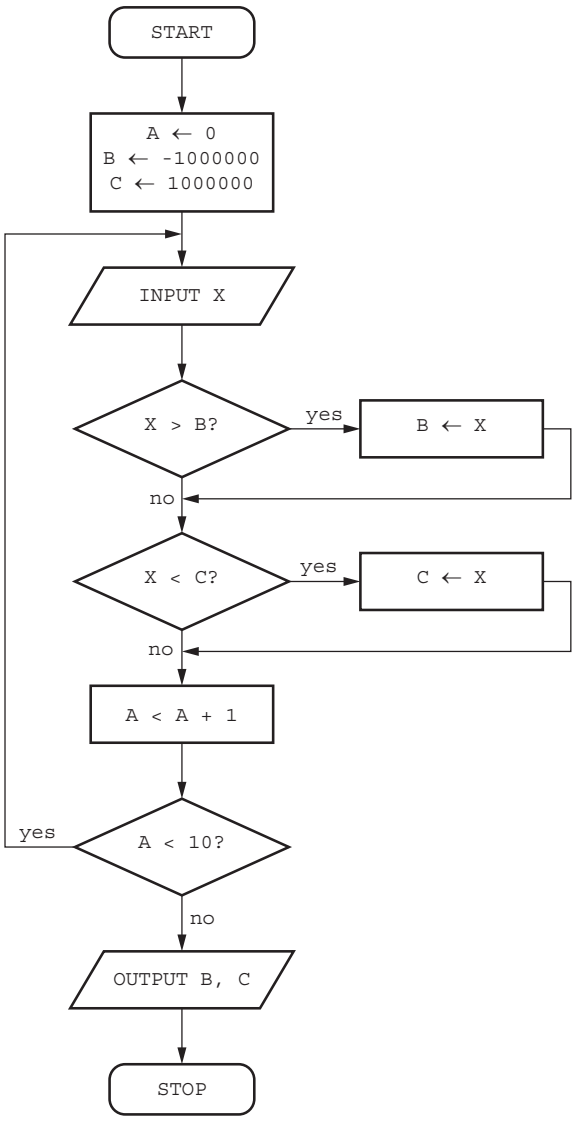
A	B	C	X	OUTPUT
0	0	100		
1	400		400	
2	800		800	
3			190	
4			170	
5			300	
6			110	
7			600	
8			150	
9			130	
10	900		900	
				900 100

There is an error as the smallest number, 110, has not been identified.

Activity 7.15

Use a trace table and some negative test data to record another dry run of the pseudocode or flowchart. What error have you found?

As this algorithm only works for numbers between 0 and 100; a better algorithm could look like this:



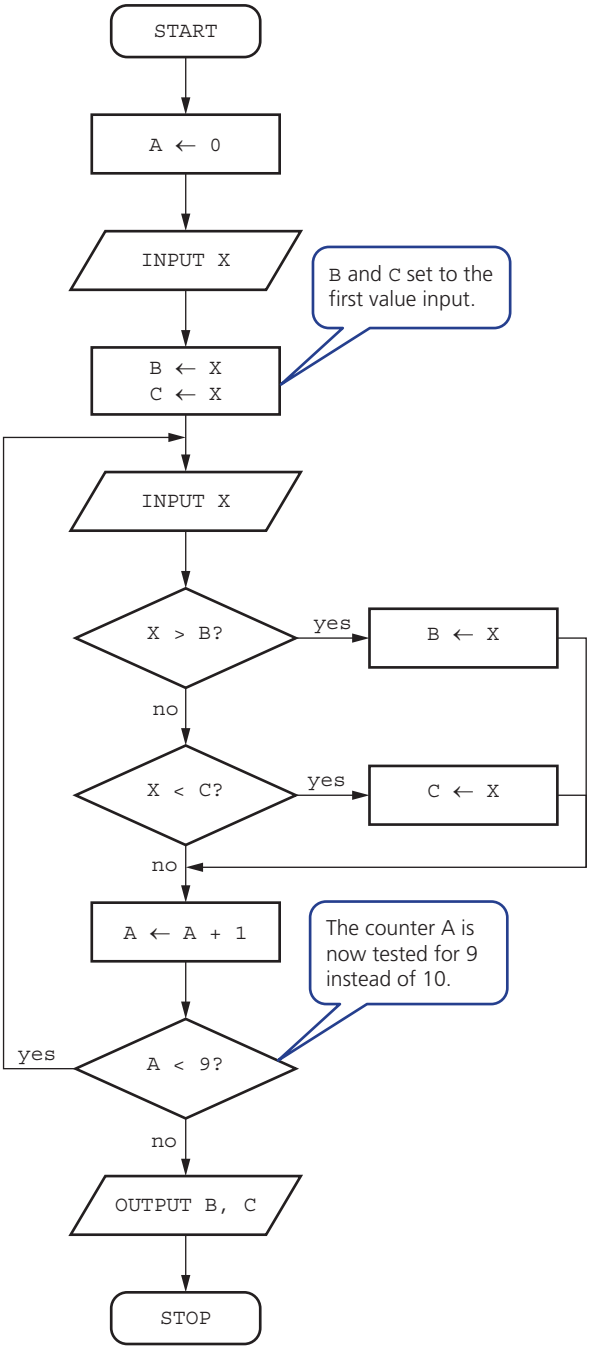
▲ **Figure 7.16** A better algorithm

This algorithm is very similar and works for a much larger range of numbers, but it still does not work for every set of numbers.

Activity 7.16

Identify two numbers where the algorithm will still fail.

In order to work for any set of numbers, the algorithm needs to be re-written to allow the largest and smallest numbers to be tested against numbers that appear in any list provided. The provisional values set at the start of the algorithm need to be chosen from the list. A standard method is to set both these provisional values to the value of the first item input.



▲ **Figure 7.17** A much better algorithm

Activity 7.17

Rewrite the pseudocode so it works for every set of numbers like the flowchart above.

Test your pseudocode algorithm with this set of test data:

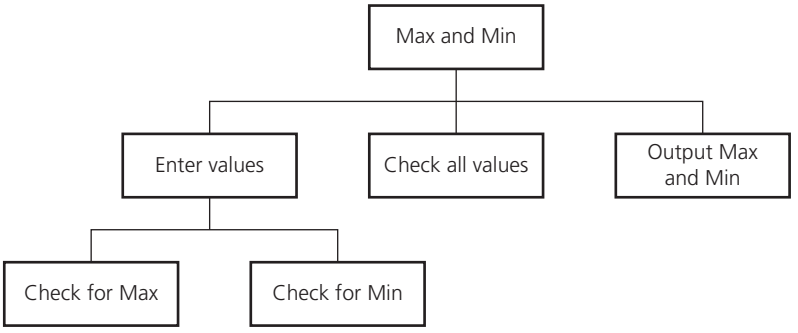
-97, 12390, 0, 77, 359, -2, -89, 5000, 21, 67

7.9 Writing and amending algorithms

There are a number of stages when producing an algorithm for a given problem:

1. Make sure that the problem is clearly specified – the purpose of the algorithm and the tasks to be completed by the algorithm.
2. Break the problem down in to sub-problems; if it is complex, you may want to consider writing an algorithm for each sub-problem. Most problems, even the simplest ones can be divided into:
   - Set up processes
   - Input
   - Processing of data
   - Permanent storage of data (if required)
   - Output of results.
3. Decide on how any data is to be obtained and stored, what is going to happen to the data and how any results are going to be displayed.
4. Design the structure of your algorithm using a structure diagram.
5. Decide on how you are going to construct your algorithm, either using a flowchart or pseudocode. If you are told how to construct your algorithm, then follow the guidance.
6. Construct your algorithm, making sure that it can be easily read and understood by someone else. Precision is required when writing algorithms, just as it is when writing program code. This involves setting it out clearly and using meaningful names for any data stores. Take particular care with conditions used for loops and selection, for example 'Counter >= 10' rather than 'Counter ten or over'. The algorithms that you have looked at so far in this chapter were not designed with readability in mind because you needed to work out what the problem being solved was.
7. Use several sets of test data (Normal, Abnormal and Boundary) to dry run your algorithm and show the results in trace tables, to enable you to find any errors.
8. If any errors are found, correct them and repeat the process until you think that your algorithm works perfectly.

Have a look at this structure diagram and flowchart for the algorithm to select the largest, Max, and smallest, Min, numbers from a list of ten numbers. This time the flowchart is more easily readable than the structure chart:



▲ **Figure 7.18** Structure chart for Max and Min